

Juan Sebastian Parra Avendaño
Nicolás Hernández Estrada
Juan David Tellez Moreno

Quiz 3 Análisis

Representación interna

Para la clase Snake se siguió la idea de un arreglo circular:

- Cuerpo: Un arreglo nativo de tipo `Position[]` que representa las posiciones de cada parte del snake.
- Size: Un entero que lleva el conteo de la cantidad actual de posiciones lógicas ocupadas por el cuerpo del snake.
- Capacity: Un entero que indica el tamaño total físico reservado actualmente en el arreglo cuerpo.
- Tail: Un puntero entero que indica el índice físico específico dentro del arreglo donde está la cola de la serpiente.
- ResizeFlag: Para poder saber si se hizo o no resize en el último addhead, para la impresión solicitada en el quiz.

Invariantes

Dynamic Array:

$$0 \leq size \leq capacity$$

$$data.length == capacity$$

$$tail = data[size-1]$$

Como la clase Snake es un arreglo circular en esencia, se tienen los siguientes invariantes:

$$0 \leq size \leq capacity$$

$$0 \leq tail < capacity$$

$$Phys(i) = (i + tail)(mod\ capacity)$$

Análisis de complejidad

Dynamic array:

Método	Complejidad temporal
size	O(1)
capacity	O(1)
get(index)	O(1)
set(index,value)	O(1)
append(value)	O(n)
removeLast()	O(1)
resize(newCapacity)	O(n)

Snake:

Método	Complejidad temporal implementada	Complejidad temporal forma ineficiente
size	$O(1)$	$O(1)$
capacity	$O(1)$	$O(1)$
get(index)	$O(1)$	$O(1)$
addHead(posicion)	$O(n)$ (Amortizado $O(1)$)	$O(n)$ (Amortizado $O(1)$)
removeTail()	$O(1)$	$O(n)$
resize(newCapacity)	$O(n)$	$O(n)$

Demostración del análisis amortizado

Para la implementación del Snake al mirar el método addHead(Postiion p) se pueden dar dos escenarios que en términos de complejidad son totalmente diferentes:

- El mejor escenario (costo 1) : la mayoría de veces cuando la serpiente realiza un movimiento , dentro del arreglo circular definido para su estructura se cuenta con casillas vacias . lo cual conlleva a que el cálculo del nuevo índice de la forma $(tail + size) \% capacity$ y el posterior almacenamiento de la cabeza en este índice tome solo 1 una operacion, lo que nos da un costo constante.
- El peor escenario (costo n) : cuando , en algunos casos la serpiente esta llena ($size == capacity$) por lo cual se debe redimensionar el tamaño de la serpiente, para ello toca crear un nuevo arreglo y copiar cada una de las posiciones viejas y acomodarlas, una a una en las posiciones del nuevo arreglo. esto tiene un costo n.

Si se hiciera un análisis plano de este código se podría concluir que el programa tiene una complejidad de costo n , pero aquí es donde entra en juego el análisis amortizado en donde se pregunta ¿Qué tan seguido ocurre este redimensionamiento ? y si esto afecta la complejidad en general del programa durante su ciclo de vida de ejecución.

Para demostrar la eficiencia de la operacion addHead se puede aplicar el metodo del Banquero en donde se tienen que definir primero las siguientes variables para la demostración:

- c_i : El costo real de la operación i.
- $\hat{c}_i$: El costo amortizado (tarifa asignada)
- C_i : El balance de créditos acumulados en la estructura tras la operación i, donde exigimos como invariante que $C_i \geq 0$ para toda secuencia de tamaño m.

ahora establecemos un “costo amortizado” constante $\hat{c}_i = 3$ para cada operación `addHead`. El costo real de una inserción en el arreglo circular, cuando no toca hacer un redimensionamiento es estrictamente $c_i = 1$.

Por definición de la ecuación contable de amortización:

$$\hat{c}_i = c_i + \Delta C$$

Al realizar una inserción ordinaria sin *resize*, la operación genera un superávit de créditos:

$$\Delta C = \hat{c}_i - c_i = 3 - 1 = +2 \text{ créditos}$$

Supongamos que la estructura ejecuta un *resize* y duplica su capacidad física de una capacidad anterior K a una nueva capacidad $C = 2K$. En el instante inmediatamente posterior al redimensionamiento, el balance de créditos específicos de expansión se encuentra en cero.

Para que la estructura vuelva a saturar su capacidad física actual C y se de el siguiente evento de redimensionamiento $O(n)$, es una condición necesaria que la serpiente experimente un crecimiento mediante la “captura” de las manzanas (que en el programa se ven representadas con un 2 en el tablero). Debido a la geometría del factor de crecimiento por duplicación ($2C$), el número mínimo de operaciones de inserción requeridas para llenar la capacidad libre del arreglo es exactamente:

$$\text{Inserciones requeridas} = C - C/2 = C/2$$

Como cada una de estas $C/2$ operaciones ordinarias inyecta de forma constante +2 créditos a nuestra bolsa de créditos, el crédito acumulado en el fondo común justo antes del desbordamiento (cuando se va a realizar una operación de *resize*) es:

$$C \text{ acumulado} = C/2 \times 2 = C \text{ créditos}$$

Cuando finalmente se ejecuta la operación $C/2 + 1$ y el tamaño transitorio supera la capacidad, se dispara el método *resize*. Según nuestro modelo de costos, transferir los elementos al nuevo arreglo nativo implica un **costo real** de $c_i = C$ operaciones de copia.

Consumimos los créditos acumulados para absorber el impacto del ciclo de copiado:

$$\hat{c}_i = c_i - C \text{ acumulado} = C - C = 0$$

Dado que el balance de créditos acumulados C_i se mantiene estrictamente ≥ 0 a lo largo de cualquier secuencia arbitraria, queda demostrado formalmente que el costo real acumulado

está acotado superiormente por el costo amortizado total, garantizando que cada operación individual addHead opera con una complejidad asintótica de **$O(1)$ amortizado**.

Comparación de las políticas de crecimiento

La principal diferencia entre las operaciones radica en cuándo se hace el resize y que tan frecuente se hace.

Nuestra política original es la B, la cual dobla la capacidad. Es el estándar y la forma que hemos trabajado en clase.

La A aumenta el tamaño del arreglo en 1, esto hace que el redimensionamiento y por consecuencia la copia de todos los elementos se tenga que hacer siempre que se hace una inserción, por lo tanto su complejidad es mayor.

Con la C tenemos un factor distinto de crecimiento, no es de 2, sino el techo de 1.5, esto hace que tenga que hacer resize un poco más frecuente, pero como sigue siendo de la forma nC , entonces no cambia la complejidad del redimensionamiento con una secuencia de n operaciones.

Tabla correspondiente a la Pregunta 5

Política	# Resizes	Total copias	Costo amortizado por crecimiento	Espacio sin usar tras resize
C+1	$n-2$	$\frac{n^2-n}{2} - 1$	$O(n)$	0
2C	$\lceil \log_2 n \rceil$	$2^{(\lceil \log_2 n \rceil)+1} - 1$	$O(1)$	50%
$\lceil 1.5C \rceil$	$\lceil \log_{1.5} n \rceil$	$4(1.5^{(\lceil \log_{1.5} n \rceil)+1} - 1)$	$O(1)$	33%